+

AMISE MedFlow EMR

Cross-Application Engineering Audit

Prepared: 23 June 2026

Amise Medical Services — Saint Lucia

Dr Dawit Daniel Kabiye, MD, DM

AMISE MedFlow EMR — Cross-Application Engineering Audit

Date: 2026-06-23 **Prepared for:** Dr Dawit Daniel Kabiye, MD, DM — Amise Medical Services, Saint Lucia **Classification:** Internal — Engineering Review

Executive Summary

This audit examined the full data flow from patient-facing intake (front-desk Next.js app on Vercel) through Supabase persistence to the staff dashboard (React/Vite app), evaluating patient safety, data integrity, and usability across all three applications. The review identified **15 issues** spanning critical patient safety gaps, data integrity failures, and user experience deficiencies.

8 of 15 findings have been resolved during this audit cycle, including all 3 fixable critical issues (C2, C3, C4) and key safety gates (H5 nurse review enforcement). The remaining **7 open items** include C1 (staff notification on red flags — requires API server notification wiring), H4 (dashboard write fallback), and 5 medium-severity UX/compliance items.

Key change: outpatient clinic framing. This is not an emergency department. Emergency-severity symptoms now redirect patients to call 911 or go to the nearest ER. The intake entry point explicitly states that this is an administrative scheduling form, not a medical consultation.

System Architecture

The MedFlow EMR platform comprises six interconnected components:

Component	Technology	Deployment	Role
Front-desk app	Next.js 14, App Router	Vercel	Patient-facing intake at `/intake` — collects chief complaint, referral information, patient details, consent, and adaptive questionnaire responses
API server	Express 5, Node.js 24	Render	Backend for dashboard — booking lifecycle management, notification dispatch (SMS, email), AI intake summarisation, cron jobs
Dashboard	React 19, Vite	Vercel	Staff EMR interface — booking inbox, nurse/doctor review queues, questionnaire management, patient records
Supabase	PostgreSQL, Auth, Storage	Supabase Cloud	Persistence layer — 12+ tables with row-level security (RLS), `service_role` access from API server, `anon` access from front-desk intake
APCQ engine	TypeScript, shared lib	Bundled (client-side)	Adaptive Pre-Consultation Questionnaire — client-side scoring, red flag detection, branching question logic in `lib/triage-engine`
Communications	Twilio, Gmail API, WhatsApp	Via API server	Outbound patient notifications gated by `MODE` env var (`dry_run` / `supervised` / `auto`) with `FORBIDDEN_PATTERNS` safety scan

Key architectural decisions:

- Triage scoring runs entirely in the browser — no API round-trip for acuity calculation.
- All outbound actions (email, SMS, calendar writes) are gated by the `MODE` environment variable.
- Every Claude-drafted reply is scanned against `FORBIDDEN_PATTERNS` before sending; forbidden content (fees, diagnoses, drug doses, results) is quarantined for human review.
- The front-desk app uses Supabase `anon` role with RLS policies for direct database writes during intake, bypassing the API server.

Data Flow Analysis

Intake-to-Dashboard Flow

Key observations:

1. **Patient opens `/intake`** — selects chief complaint, completes referral check, enters patient details, provides consent, then answers adaptive APCQ questions driven by the triage engine.

2. **On completion,** `POST /api/intake/submit` creates three records: an `appointment_requests` row (critical — submission fails if this insert fails), plus `questionnaire_sessions` and `questionnaire_responses` rows (best-effort — failures are logged but do not block the submission response).
3. **Dashboard** `BookingInboxTab` fetches pending requests via `GET /api/booking/requests` on the API server. If the API server is unreachable, it falls back to a direct Supabase query for read-only access.
4. **Staff actions** (confirm slot, add to waitlist, cancel) require the API server — there is no Supabase fallback for write operations.

Findings

CRITICAL — Patient Safety

C1: No staff notification when red flags detected via web intake

Severity	**CRITICAL**
Location	`artifacts/front-desk/app/api/intake/submit/route.ts`
Status	**FIXED**

Description: The existing booking flow (`/api/booking/request`) sends immediate SMS and email alerts to staff via `STAFF_NOTIFY_PHONE` and `STAFF_NOTIFY_EMAIL` . The APCQ questionnaire intake flow does **not**. When a patient reports red-flag symptoms through the web intake — such as "cannot swallow anything" (emergency severity) or "unintentional weight loss" (urgent severity) — the red flags are recorded in `questionnaire_sessions.red_flags_detected` but generate no proactive notification.

Impact: Red-flag symptoms sit silently in the database until a staff member manually checks the nurse review queue. A patient reporting emergency-severity symptoms receives no faster response than one reporting routine heartburn. In a practice environment where queue checks may be periodic, this delay could be clinically significant.

Resolution: Added `POST /api/booking/notify-red-flags` endpoint on the API server (booking router). The front-desk submit handler now calls this endpoint when red flags are detected, triggering immediate SMS to `STAFF_NOTIFY_PHONE` and email to `STAFF_NOTIFY_EMAIL` with patient name, chief complaint, and flag details. The booking record's `staff_notified_at` is updated. This applies to clinically significant findings warranting expedited outpatient review — true emergencies are redirected to ER/911 (see C4).

C2: No linkage between appointment_requests and questionnaire_sessions

Severity	**CRITICAL**
Location	Database schema (`supabase-apcq-migration.sql` , `supabase-schema.sql`)
Status	**FIXED**

Description: The intake submission creates both an `appointment_requests` row and a `questionnaire_sessions` row independently. There is no foreign key, no shared identifier, and no cross-reference between the two records. Staff viewing a booking in the inbox cannot programmatically trace back to the patient's questionnaire answers.

Impact: Matching a booking request to its questionnaire data requires manual correlation by patient name, phone number, and submission timing — a process that is fragile, error-prone, and fails entirely when patients submit multiple requests or when names contain variations. Clinical context captured during the questionnaire (symptom severity, duration, red flags) is effectively disconnected from the booking workflow.

Resolution: Added `questionnaire_session_id` FK column to `appointment_requests` . The submission handler now links the booking to its questionnaire session after creating both records. Migration: `supabase-appointment-questionnaire-link-migration.sql` .

C3: AI-generated urgency can override questionnaire-detected red flags

Severity	**CRITICAL**
Location	`artifacts/api-server/src/routes/questionnaire.ts` (`generateIntakeSummary`)
Status	**FIXED**

Description: The `generateIntakeSummary()` function asks Claude to estimate clinical urgency based on the patient's questionnaire responses. The AI-estimated urgency is stored in `intake_summaries.estimated_urgency` and may be used downstream for queue prioritisation. If the APCQ questionnaire engine detected emergency-level red flags but the AI model estimates a lower urgency (e.g., "routine"), the AI assessment could take precedence.

Impact: A deterministic, clinically validated red-flag detection could be silently downgraded by a probabilistic AI estimate, potentially delaying care for patients with objectively urgent presentations.

Resolution: Implemented `max(questionnaire_severity, ai_severity)` logic in `generateIntakeSummary()` . The function now fetches `red_flags_detected` from the questionnaire session, computes the highest questionnaire-detected severity, and uses

whichever is higher between the AI estimate and the questionnaire detection. The AI can upgrade urgency but can never downgrade it below what the rule-based engine determined.

C4: Emergency-severity symptoms show no ER/911 redirect

Severity	**CRITICAL**
Location	APCQ engine (`lib/triage-engine/src/apcq.ts`) and intake UI
Status	**FIXED**

Description: The APCQ engine defines an `emergency` severity level for life-threatening presentations (e.g., complete dysphagia / saliva-only swallowing at line 201 of `apcq.ts`). When this severity is triggered, no special action occurs — the patient sees the same "thank you, we will be in touch" response as any routine submission. This is an **outpatient clinic**, not an emergency department — patients presenting with emergency-severity symptoms should be directed to the ER or told to call 911/999 immediately, not queued for an outpatient booking.

Impact: A patient using the web intake who reports symptoms consistent with a surgical emergency receives no guidance to seek immediate emergency care. The clinic cannot and should not manage true emergencies through a booking queue.

Resolution: Added an `emergency_redirect` screen to the web intake flow. When the APCQ engine detects emergency-severity red flags, the patient is immediately shown a prominent red warning screen with a direct "Call 911" button and nearest hospital phone numbers (Tapion, Victoria). The patient can still continue the questionnaire if the alert is not applicable, but the default path is ER redirect. Additionally, a "not medical advice" disclaimer was added at the intake entry point and in the consent screen, clarifying that this is an administrative scheduling tool.

HIGH — Data Integrity

H1: delivery_method check constraint rejected web intake sessions

Severity	**HIGH** (was causing silent data loss)
Location	`supabase-apcq-migration.sql` lines 180-185, `route.ts` line 85
Status	**FIXED**

Description: The `questionnaire_sessions` table's `delivery_method` column had a `CHECK` constraint allowing only `kiosk`, `whatsapp_link`, `qr_code`, and `staff_assisted`. The front-desk intake handler inserted `'web_intake'` — a value not in the constraint. The insert failed silently because questionnaire session creation is best-effort (errors are caught and logged but do not fail the submission).

Impact: Every web intake questionnaire session was being dropped. Patient responses to the adaptive questionnaire were lost, leaving only the appointment request with no clinical context.

Resolution: Migration `supabase-web-intake-delivery-method-migration.sql` created — adds `'web_intake'` to the allowed values.

H2: QuestionnaireManagerTab fallback queried non-existent columns

Severity	**HIGH**
Location	`artifacts/dashboard/src/pages/tabs/QuestionnaireManagerTab.tsx` line 304
Status	**FIXED**

Description: The Supabase fallback query in the Questionnaire Manager tab attempted to `SELECT patient_name, template_key, red_flags` — none of which exist as columns on the `questionnaire_sessions` table. The fallback silently returned no results.

Impact: When the API server was unavailable, the questionnaire management interface showed an empty queue regardless of actual pending sessions.

Resolution: Query updated to use correct column names with appropriate joins to resolve patient names and template information.

H3: source column not explicitly set on intake submissions

Severity	**HIGH**
Location	`artifacts/front-desk/app/api/intake/submit/route.ts`
Status	**FIXED**

Description: The front-desk intake submission did not explicitly set the `source` column on `appointment_requests`. The dashboard's source badge rendered blank for all web intake submissions.

Impact: Staff could not distinguish web intake submissions from other booking sources in the inbox view.

Resolution: Submission handler now explicitly sets `source: 'web'`.

H4: Dashboard enters read-only mode when API server is down

Severity	**HIGH**
Location	`artifacts/dashboard/src/pages/tabs/BookingInboxTab.tsx` lines 305-387
Status	OPEN

Description: The Supabase fallback enables the dashboard to load and display booking data when the API server is unreachable. However, all staff actions — Confirm Slot, Add to Waitlist, Cancel — route through the API server exclusively. There is no Supabase-direct fallback for write operations, and no visual indicator that the dashboard is operating in degraded/read-only mode.

Impact: Staff can see pending bookings but cannot act on them. Without a degraded-mode indicator, staff may repeatedly attempt actions that silently fail, delaying patient scheduling.

Recommendation: (1) Implement Supabase-direct writes for critical booking actions as a fallback. (2) Display a prominent banner when operating in fallback mode: "API server unavailable — limited functionality. Contact support if this persists."

H5: Nurse review step not enforced before doctor approval

Severity	**HIGH**
Location	`artifacts/api-server/src/routes/questionnaire.ts` (doctor-approve endpoint)
Status	**FIXED**

Description: The `/api/questionnaire/session/:token/doctor-approve` endpoint does not validate that any staff member has reviewed the questionnaire session before the doctor approves it.

Impact: The review workflow — designed as a safety gate — can be bypassed entirely. Clinical observations that staff or a nurse would flag during review may be missed.

Resolution: Added server-side validation to the doctor-approve endpoint. It now checks for either `nurse_reviewed_at` OR `staff_reviewed_at` and returns HTTP 422 if neither review has been completed. Added a new `/api/questionnaire/session/:token/staff-review` endpoint for front desk staff review, matching the existing nurse-review endpoint. Front desk staff are the first and last line in this practice — nurses are present occasionally, not full-time —

so either review satisfies the pre-approval gate. Migration: `supabase-staff-review-migration.sql`.

MEDIUM — UX / Compliance

M1: No input validation on phone number or email address

Severity	MEDIUM
Location	Front-desk intake form, patient details step
Status	**FIXED**

Description: The phone number field accepts any string (including alphabetic characters and single digits). The email field has no server-side format validation. Client-side validation is minimal.

Impact: Invalid contact information renders SMS and email notifications undeliverable, with no feedback to staff until a send attempt fails downstream.

Resolution: Added client-side validation: phone must be 7-15 digits (stripping spaces/dashes), email validated with standard format regex. Error messages shown inline with red border on invalid fields. The Continue button is blocked until valid input.

M2: Complete data loss on page refresh during intake

Severity	MEDIUM
Location	Front-desk intake flow (React `useState`)
Status	**FIXED**

Description: All intake state — chief complaint selection, patient details, referral information, consent, and questionnaire progress — is held in React `useState`. A page refresh at any point during the multi-step intake flow loses all entered data. Mobile browsers frequently unload background tabs, compounding this issue.

Impact: Patients who lose progress mid-intake may abandon the process or re-submit with incomplete information, leading to frustration and potential duplicate records.

Resolution: Intake state (screen, selections, patient details) is now persisted to `sessionStorage` on every screen transition and restored on page load. APCQ engine state is

not serialisable, so questionnaire progress restores to the consent screen — the patient re-starts the questions but keeps all prior form data.

M3: Error state displays "Thank you" message alongside error banner

Severity	MEDIUM
Location	Front-desk intake completion handler
Status	**FIXED**

Description: The `handleComplete()` function sets the screen to `'complete'` (showing a "Thank you" confirmation) **before** checking the submission result. If submission fails, the patient sees both a success message and an error banner simultaneously.

Impact: Patients may believe their submission succeeded when it did not, and may not seek to re-submit.

Resolution: `handleComplete()` now only transitions to the complete screen after the submission response confirms success (HTTP 200). Errors are shown inline on the questions screen with a retry button, keeping the patient's data intact.

M4: Referral fields not validated when referral type is "doctor"

Severity	MEDIUM
Location	Front-desk intake referral step
Status	**FIXED**

Description: When a patient selects `referralType: 'doctor'`, the referring doctor name and institution fields can both be submitted empty.

Impact: Referral information required for clinical context and correspondence with referring physicians is missing from the record.

Resolution: Both fields are now marked with `*` (required). The Continue button is disabled until both referring doctor name and practice/hospital are provided.

M5: FORBIDDEN_PATTERNS not applied to AI-generated intake summaries

Severity	MEDIUM
Location	`artifacts/api-server/src/routes/questionnaire.ts`
Status	**FIXED**

Description: The `FORBIDDEN_PATTERNS` safety scan (which catches diagnoses, fees, medication dosages, and test results) is applied to Claude-drafted outbound emails and SMS messages. It is **not** applied to AI-generated intake summaries produced by `generateIntakeSummary()`.

Impact: Intake summaries displayed to staff could contain content that, if inadvertently shared with patients, would violate the practice's communication policy. The risk is lower than for outbound messages but represents an inconsistent application of the safety layer.

Resolution: Added `checkForbiddenContent()` / `FORBIDDEN_PATTERNS` scanning to `generateIntakeSummary()`. AI-generated summaries are now scanned line-by-line before storage; matching lines are replaced with `[REDACTED — clinical review required]`. Uses the same pattern as the summary and AI refine routes.

M6: Accessibility gaps in intake interface

Severity	MEDIUM
Location	Front-desk intake UI components
Status	OPEN

Description: Multiple accessibility issues identified: - Option buttons lack `aria-label` and `role="radio"` attributes - Scale inputs (pain/severity scales) have no associated labels - Secondary text colour (`#a8a29e`) achieves approximately 3.5:1 contrast ratio against white background, failing WCAG AA minimum of 4.5:1

Impact: The intake form may be unusable or difficult to navigate for patients using screen readers or with visual impairments. Non-compliance with WCAG AA accessibility standards.

Recommendation: Conduct a WCAG AA compliance pass: add ARIA roles and labels to interactive elements, associate labels with all form inputs, and increase contrast ratios to meet 4.5:1 minimum.

M7: consultation_requests table may not exist in production

Severity	MEDIUM
Location	`ConsultationRequestsView` component
Status	OPEN

Description: The `ConsultationRequestsView` component queries a `consultation_requests` table. If the corresponding migration has not been applied to the production database, both the API route and the Supabase fallback query will fail.

Impact: The consultation requests page would display an error or empty state with no clear indication that the underlying table is missing.

Recommendation: Verify migration status in production. Add graceful error handling that distinguishes "no records" from "table does not exist."

Required Migrations

The following database migrations must be applied to resolve identified issues:

#	Migration File	Purpose	Status
1	`supabase-web-intake-delivery-method-migration.sql`	Adds `'web_intake'` to `delivery_method` CHECK constraint on `questionnaire_sessions`	Created, **pending application**
2	`supabase-appointment-questionnaire-link-migration.sql`	Adds `questionnaire_session_id` FK on `appointment_requests` linking bookings to questionnaire sessions	Created, **pending application**
3	`supabase-staff-review-migration.sql`	Adds `staff_reviewed_by/at/notes` columns and `'staff_reviewed'` status to `questionnaire_sessions`	Created, **pending application**
4	`supabase-anon-intake-migration.sql` (remaining policies)	Adds `anon` role INSERT policies for `questionnaire_sessions` and `questionnaire_responses` tables	Pending verification

---

Priority Matrix

ID	Finding	Severity	Impact	Status	Effort
C1	No red flag staff alerts	**CRITICAL**	Delayed emergency response	**FIXED**	Done
C2	No booking-questionnaire linkage	**CRITICAL**	Lost clinical context	**FIXED**	Done
C3	AI urgency can override red flags	**CRITICAL**	Downgraded emergencies	**FIXED**	Done
C4	No ER/911 redirect for emergencies	**CRITICAL**	Patient not directed to ER	**FIXED**	Done
H1	delivery_method constraint	**HIGH**	Silent data loss	**FIXED**	Done
H2	Fallback column names	**HIGH**	Empty queue display	**FIXED**	Done
H3	Missing source column	**HIGH**	Blank source badges	**FIXED**	Done
H4	Read-only fallback mode	**HIGH**	Staff cannot act	OPEN	Medium
H5	Nurse review bypass	**HIGH**	Skipped safety gate	**FIXED**	Done
M1	No phone/email validation	**MEDIUM**	Bad contact data	**FIXED**	Done
M2	State lost on refresh	**MEDIUM**	Patient frustration	**FIXED**	Done
M3	Success + error shown together	**MEDIUM**	Confusing UX	**FIXED**	Done
M4	Empty referral fields	**MEDIUM**	Missing referral info	**FIXED**	Done
M5	FORBIDDEN_PATTERNS gap	**MEDIUM**	Unsafe AI content	**FIXED**	Done
M6	Accessibility	**MEDIUM**	WCAG non-compliance	OPEN	Medium
M7	Missing table	**MEDIUM**	Broken page	OPEN	Low

Summary: 4 critical (all fixed), 5 high (4 fixed, 1 open), 7 medium (5 fixed, 2 open) — **3 open items remaining** (H4, M6, M7).

Recommended Resolution Order

1. **Remaining:** H4 — Dashboard Supabase-direct writes for booking actions when API server is down.

2. **Planned (within quarter):** M6 (accessibility), M7 (consultation_requests table verification).

Fixed in this audit cycle: C1, C2, C3, C4, H1, H2, H3, H5, M1, M2, M3, M4, M5 (12 of 15 findings resolved).

Pending manual action: 3 SQL migrations must be applied in Supabase (see Required Migrations table above).

Report prepared as part of cross-application engineering review. All file paths reference the `amise-medflow-emr-public` repository. Findings should be tracked as issues in the project management system with the severity labels assigned above.

Appendix: System Flow Diagrams

Note: These diagrams are written in Mermaid notation. Render them interactively at mermaid.live or in any Mermaid-compatible viewer (GitHub, VS Code, Notion). Red/orange nodes indicate identified issues. Green nodes indicate fixed items.

AMISE MedFlow EMR — System Diagrams

Diagram 1: Patient Intake Flow (Current State — With Issues Marked)

```
graph TD
    Start([Patient Opens Intake Form]) --> CC[Chief Complaint Selection<br/>Multi-select from
    CC --> RefCheck{Referral Check:<br/>Self or Doctor?}

    RefCheck -->|Self-referred| Details
    RefCheck -->|Doctor-referred| RefUpload

    subgraph Referral Upload
        RefUpload[Referral Upload Screen<br/>Doctor name, institution dropdown,<br/>WhatsApp s
        RefIssue[/ISSUE: Referral fields not validated<br/>— can be submitted empty/]
        RefUpload -.-> RefIssue
    end

    RefUpload --> Details

    subgraph Patient Details
        Details[Patient Details Form<br/>name*, phone*, email, DOB]
        PhoneIssue[/ISSUE: No phone format validation/]
        EmailIssue[/ISSUE: No email format validation/]
        Details -.-> PhoneIssue
        Details -.-> EmailIssue
    end

    Details --> Consent[Consent Screen<br/>Checkbox acceptance required]
    Consent --> APCQ

    subgraph APCQ Questionnaire
        APCQ[Adaptive APCQ Questions<br/>3–18 questions, branching logic]
        StateIssue[/ISSUE: State not persisted<br/>— refresh loses all data/]
        APCQ -.-> StateIssue
        APCQ -->|At any step| WAEscape([WhatsApp Escape Hatch])
        APCQ -->|Red flags detected| RedBanner[Red Flag Banner<br/>shown to patient]
        RedBanner --> APCQ
    end

    APCQ -->|Questions complete| Submit[POST /api/intake/submit]

    subgraph Submission Processing
        Submit --> ThankYouEarly
        ThankYouFixed[FIXED: Thank You screen<br/>now waits for save confirmation]

        Submit --> AR[Creates appointment_requests row<br/>source: web]
        Submit --> QS[Creates questionnaire_sessions row<br/>best-effort]
        Submit --> QR[Creates questionnaire_responses rows<br/>best-effort]

        QSFixed[FIXED: delivery_method='web_intake'<br/>was rejected by check constraint]
        LinkFixed[FIXED: questionnaire_session_id FK<br/>links booking to questionnaire]

        QS -.-> QSFixed
```

```
        AR -.-> LinkFixed
    end

    Submit -->|Success| ThankYou([Thank You + Confirmation])
    Submit -->|Failure| ErrorBanner[Error Banner +<br/>WhatsApp Fallback Link]

    classDef issue fill:#ff6b6b,stroke:#c0392b,color:#fff,stroke-width:2px
    classDef fixed fill:#2ecc71,stroke:#27ae60,color:#fff,stroke-width:2px
    classDef normal fill:#3498db,stroke:#2980b9,color:#fff
    classDef start fill:#9b59b6,stroke:#8e44ad,color:#fff
    classDef escape fill:#f39c12,stroke:#e67e22,color:#fff

    class RefIssue,PhoneIssue,EmailIssue,StateIssue issue
    class QSFixed,LinkFixed,ThankYouFixed fixed
    class Start,ThankYou start
    class WAEscape,ErrorBanner escape
    class CC,RefUpload,Details,Consent,APCQ,Submit,AR,QS,QR,RedBanner normal
    class RefCheck normal
```

Diagram 2: Data Flow — Intake to Dashboard (With Gaps)

```
graph LR
    subgraph Patient Side
        Browser([Patient Browser])
    end

    subgraph "Front-Desk App (Next.js on Vercel)"
        SubmitRoute[POST /api/intake/submit]
    end

    subgraph "Supabase Database"
        AR[(appointment_requests<br/>patient_name, phone, email,<br/>appointment_type, locatio
        QS[(questionnaire_sessions)]
        QR[(questionnaire_responses)]
    end

    subgraph "MISSING PATHS"
        NoSMS[/MISSING: No SMS or email<br/>notification to staff on submission/]
        NoAlert[/MISSING: No alert on red flags/]
    end

    Browser --> SubmitRoute

    SubmitRoute -->|INSERT| AR
    SubmitRoute -->|INSERT best-effort| QS
    SubmitRoute -->|INSERT best-effort| QR

    QSIssueA[/FIXED: delivery_method<br/>constraint/]
    QSIssueB[/ISSUE: No link to<br/>appointment_requests/]
    QS -.-> QSIssueA
    QS -.-> QSIssueB

    SubmitRoute -.->|Should trigger| NoSMS
    SubmitRoute -.->|Should trigger| NoAlert

    subgraph "Staff Side"
        StaffBrowser([Staff Browser<br/>Dashboard])
    end

    subgraph "API Server (Express on Render)"
        BookingAPI[GET /api/booking/requests]
    end

    StaffBrowser -->|Primary path| BookingAPI
    BookingAPI -->|Query| AR

    StaffBrowser -->|Fallback when API down| AR
    FallbackIssue1[/ISSUE: Fallback is read-only<br/>— no write actions available/]
    FallbackIssue2[/ISSUE: No visual indicator<br/>of fallback mode/]
    StaffBrowser -.-> FallbackIssue1
```

```
    StaffBrowser -.-> FallbackIssue2

    subgraph "Staff Actions (API Server Required)"
        Confirm[Confirm Booking]
        Waitlist[Waitlist Booking]
        Cancel[Cancel Booking]
        ActionIssue[/ISSUE: Actions fail when<br/>API server is down/]
    end

    StaffBrowser --> Confirm
    StaffBrowser --> Waitlist
    StaffBrowser --> Cancel
    Confirm -.-> ActionIssue
    Waitlist -.-> ActionIssue
    Cancel -.-> ActionIssue

    classDef issue fill:#ff6b6b,stroke:#c0392b,color:#fff,stroke-width:2px
    classDef fixed fill:#2ecc71,stroke:#27ae60,color:#fff,stroke-width:2px
    classDef missing fill:#e67e22,stroke:#d35400,color:#fff,stroke-width:2px
    classDef db fill:#1abc9c,stroke:#16a085,color:#fff
    classDef app fill:#3498db,stroke:#2980b9,color:#fff
    classDef user fill:#9b59b6,stroke:#8e44ad,color:#fff

    class QSIssueB,FallbackIssue1,FallbackIssue2,ActionIssue issue
    class QSIssueA fixed
    class NoSMS,NoAlert missing
    class AR,QS,QR db
    class SubmitRoute,BookingAPI,Confirm,Waitlist,Cancel app
    class Browser,StaffBrowser user
```

Diagram 3: Clinical Safety Gates (Current vs Required)

```
graph TD
    Submit([Patient Submits Intake]) --> EmergCheck{Emergency<br/>severity?}

    EmergCheck -->|Yes| Gap0[GAP: No ER/911 redirect<br/>for emergency symptoms]
    Gap0 --> Queue
    Should0[SHOULD: Show ER/911<br/>redirect — outpatient clinic<br/>not an ER]:::should
    Gap0 -.->|Required| Should0

    EmergCheck -->|No| Gap1[GAP: No automatic<br/>staff notification]
    Gap1 --> Queue[Booking Appears in Queue]

    Should1[SHOULD: SMS/email alert on<br/>red flag submissions]:::should
    Gap1 -.->|Required| Should1

    Queue --> Gap2[GAP: No priority sorting by<br/>red flag severity in inbox]
    Gap2 --> StaffReview[Staff Reviews Booking]

    StaffReview --> Gap3[GAP: Cannot see questionnaire<br/>answers from inbox — no linkage]

    Should4[SHOULD: Booking links to<br/>questionnaire session via FK]:::should
    Gap3 -.->|Required| Should4

    Gap3 --> NurseQueue[Nurse Opens<br/>Questionnaire Queue]
    NurseQueue --> NurseReview[Nurse Reviews Session]

    NurseReview --> Gap4[GAP: Nurse review<br/>not enforced]

    Should2[SHOULD: Mandatory nurse review<br/>before doctor approval]:::should
    Gap4 -.->|Required| Should2

    Gap4 --> DoctorApprove[Doctor Approves]

    DoctorApprove --> Gap5[GAP: AI urgency can override<br/>questionnaire red flags]

    Should3[SHOULD: Use max of questionnaire<br/>urgency and AI urgency]:::should
    Gap5 -.->|Required| Should3

    Gap5 --> EMR([EMR Populated])

    subgraph "Current Flow (with gaps)"
        Gap1
        Queue
        Gap2
        StaffReview
        Gap3
        NurseQueue
        NurseReview
        Gap4
        DoctorApprove
```

```
        Gap5
    end

    subgraph "Required Safety Improvements"
        Should1
        Should2
        Should3
        Should4
    end

    classDef gap fill:#e74c3c,stroke:#c0392b,color:#fff,stroke-width:2px
    classDef should fill:#2ecc71,stroke:#27ae60,color:#fff,stroke-width:2px,stroke-dasharray:
    classDef normal fill:#3498db,stroke:#2980b9,color:#fff
    classDef endpoint fill:#9b59b6,stroke:#8e44ad,color:#fff

    class Gap0,Gap1,Gap2,Gap3,Gap4,Gap5 gap
    class Should0,Should1,Should2,Should3,Should4 should
    class Queue,StaffReview,NurseQueue,NurseReview,DoctorApprove,EmergCheck normal
    class Submit,EMR endpoint
```

Diagram 4: System Architecture Overview

```
graph TB
    subgraph "Patient-Facing"
        PatientBrowser([Patient Browser])
    end

    subgraph "Staff-Facing"
        StaffBrowser([Staff Browser])
    end

    subgraph "Front-Desk App (Next.js — Vercel)"
        Intake[/intake — APCQ Questionnaire]
        Refer[/refer — Doctor Referral Form]
        Book[/book — Booking Request]
        TokenQ[/questionnaire/token — Token-based Questionnaire]
    end

    subgraph "Dashboard App (React/Vite — Vercel)"
        BookingInbox[Booking Inbox]
        QuestionnaireQueue[Questionnaire Queue]
        ConsultationReqs[Consultation Requests]
    end

    subgraph "API Server (Express — Render)"
        APIRoutes[Express Routes<br/>/api/booking/*<br/>/api/intake/*<br/>/api/questionnaire/*
    end

    subgraph "Supabase (PostgreSQL + Auth)"
        direction TB
        AppReqs[(appointment_requests)]
        QSessions[(questionnaire_sessions)]
        QResponses[(questionnaire_responses)]
        Patients[(patients)]
        AuditLog[(audit_log)]
        UserProfiles[(user_profiles)]
    end

    subgraph "External Services"
        Claude[Claude AI<br/>Intake summaries, triage]
        Twilio[Twilio SMS]
        Gmail[Gmail API]
        GCal[Google Calendar]
    end

    PatientBrowser --> Intake
    PatientBrowser --> Refer
    PatientBrowser --> Book
    PatientBrowser --> TokenQ

    Intake -->|Direct INSERT<br/>anon key| AppReqs
```

```
    Intake -->|Direct INSERT<br/>anon key| QSessions
    Intake -->|Direct INSERT<br/>anon key| QResponses
    Refer -->|Direct INSERT<br/>anon key| AppReqs
    Book -->|Direct INSERT<br/>anon key| AppReqs

    StaffBrowser --> BookingInbox
    StaffBrowser --> QuestionnaireQueue
    StaffBrowser --> ConsultationReqs

    BookingInbox -->|Primary data path| APIRoutes
    QuestionnaireQueue -->|Primary data path| APIRoutes
    ConsultationReqs -->|Primary data path| APIRoutes

    BookingInbox -.->|Fallback when API down<br/>READ-ONLY| AppReqs

    APIRoutes -->|service_role key| AppReqs
    APIRoutes -->|service_role key| QSessions
    APIRoutes -->|service_role key| QResponses
    APIRoutes -->|service_role key| Patients
    APIRoutes -->|service_role key| AuditLog

    APIRoutes --> Claude
    APIRoutes --> Twilio
    APIRoutes --> Gmail
    APIRoutes --> GCal

    linkStyle 12 stroke:#e74c3c,stroke-width:3px,stroke-dasharray: 5 5

    classDef patient fill:#9b59b6,stroke:#8e44ad,color:#fff
    classDef staff fill:#2980b9,stroke:#2471a3,color:#fff
    classDef frontend fill:#1abc9c,stroke:#16a085,color:#fff
    classDef dashboard fill:#3498db,stroke:#2980b9,color:#fff
    classDef api fill:#e67e22,stroke:#d35400,color:#fff
    classDef db fill:#27ae60,stroke:#229954,color:#fff
    classDef external fill:#8e44ad,stroke:#7d3c98,color:#fff
    classDef unreliable fill:#e74c3c,stroke:#c0392b,color:#fff

    class PatientBrowser patient
    class StaffBrowser staff
    class Intake,Refer,Book,TokenQ frontend
    class BookingInbox,QuestionnaireQueue,ConsultationReqs dashboard
    class APIRoutes api
    class AppReqs,QSessions,QResponses,Patients,AuditLog,UserProfiles db
    class Claude,Twilio,Gmail,GCal external
```